Saronic Technologies

Interview Project Design Document

Christopher Leese
October 19, 2026

Preface

I have elected to implement project 2, “Fleet Simulation Engine”. The fleet simulation engine choice is most aligned with my interests and background. To cleanly illustrate my engineering process, this design document contains a series of ordered sections with corresponding page numbers. The following table of contents lists the design document’s sections:

1. Project Decomposition 2

1.1. Preliminary Notes 3

1.2. Requirements Decomposition 17

1.2.1. Google Slides Supplemental Illustration 31

2. Implementation Plan 32

2.1. Trade Study 32

2.2. Workflow Overview 33

2.3. Verification and Validation 34

3. Post-Implementation Analysis 36

3.1. Scalability 36

3.2. Unresolved Items 37

3.3. Lessons Learned 38

This design document is written without the use of AI.
All works in this document are my own.

Project Decomposition

1.1) Preliminary Notes

Much of this section is copied over from the document provided by Saronic. **Therefore, skipping to section 1.2 is recommended if a formal requirements decomposition is of immediate interest.** The provided document has been reorganized by me to facilitate my requirements decomposition in 1.2. This breakdown helps me become familiarized with the project needs. Additionally, it gives me the opportunity to highlight important information and questions (labeled as TBResolved) which then are addressed in requirements.

Background:

- Discrete-time simulation engine
- Scale is “fleet of vessels”
- Command interface is “[entirely external commands]”
- We need to “track and report vessel state at **each** point in time.”

I/O Model:

We must:

- Read newline-delimited JSON from stdin
- Write to stdout
- Produces **exactly** one line of output **per**-command, processed in order.

Responses (outputs) are either:

- `{"status": "ok", "payload": <WorldState>}`
- `{"status": "error", "payload": {"message": "<reason>"}}`

Behavioral Rules:

Execution Order

- Commands take effect in the order received.
- Planned State is updated **before** the response is written.

Committed State vs. Planned State

- Committed State:
  - The state of the world at a tick.
  - It is a snapshot of the **planned** state when *tick* is called.
    - I.e. tells us what is the planned state during that tick
- Planned State:
  - The expected state of the world given the commands since the last tick applied to the committed state.
  - This updates immediately with each command and becomes the new committed state when *tick* is called.

Example:

If a vessel is IDLE at the last committed state, the *set_destination* can be called on that vessel as many times as the user wants. Each set_destination command for the vessel:

- **overrides** the last set_destination command
- **updates** the planned state

Only once *tick* is called, does the planned state become the new committed state, and further set_destination will fail since the vessel is in transit.

- This means prior to *tick*, we can set dest. as many times as we'd like. Upon *tick*, we move the planned state to the committed state. That is,

$$\textbf{\textit{tick();}} \Rightarrow \mathbf{S_c = S_p}$$

Tick semantics

- Each tick command advances the sim by one time step.
- For a *set_destination* issued at tick T, arrives_at_tick is defined as:

arrives_at_tick = T + travel_times[current_location][destination]

  - **travel_times := NxN matrix, N := integer**

- A vessel that arrives at tick T reports IDLE at tick T.
  - i.e. it does **not** report transit on the tick that it arrives.

arrived ? in_transit = 0 : in_transit = 1;

- A vessel reporting IDLE may be commanded in the same tick it arrives.

IDLE ? can_command = 1 : can_command = 0;

Commands:

- init — Must be the first command.
  - Returns world state at tick 0.
  - Any other command before init, or a second init, returns an error.
    - ^ use a flag!

```
{"command": "init", "parameters": {"travel_times": [[0,4,7],[4,0,3],[7,3,0]],
"vessels": [{"id": "v1", "location": 0}, {"id": "v2", "location": 2}]}}

{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "idle",
"location": 0}, "v2": {"state": "idle", "location": 2}}}}
```

  - travel_times: N×N integer matrix. travel_times[i][j] is ticks to travel from i to j.
  - Diagonal is 0; all off-diagonal values are positive integers.
  - vessels: list of {id, location}. Each vessel starts idle at the given location index.

- tick — Advances the simulation by one time step.


```
{"command": "tick", "parameters": {}}
{"status": "ok", "payload": {"tick": 1, "vessels": {"v1": {"state": "transit", "from":
0, "to": 1, "arrives_at_tick": 4}, "v2": {"state": "idle", "location": 2}}}}
```

- set_destination — Commands a vessel to travel to a location. Valid only if the committed state is idle.
    - Rejected if the vessel is in transit, vessel_id is unknown, or destination is out of range.

  TBResolved: What happens if a user commands set_destination for the vessel's current location?

```
{"command": "set_destination", "parameters": {"vessel_id": "v1", "destination": 1}}
{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "transit", "from":
0, "to": 1, "arrives_at_tick": 4}, "v2": {"state": "idle", "location": 2}}}}
```

    - On failure: `{"status": "error", "payload": {"message": "vessel v1 is not idle"}}`
    - destination: valid location index (0 to N-1)

- get_state — Returns planned state.

```
{"command": "get_state", "parameters": {}}
{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "transit", "from":
0, "to": 1, "arrives_at_tick": 4}, "v2": {"state": "idle", "location": 2}}}}
```

Follow-on 1: Cargo and Dock Ops

Changes from Base:

- New concept of cargo and docks to the sim
  - Each location has a dock
    - Dock maintains an inventory of cargo
    - Dock has dock_throughput
      - dock_throughput := maximum number of cargo operations that a dock can process in 1 tick();
    - Cargo is represented by its destination
      - there is **no** distinction between two cargo with the **same** destination
  - Vessels load from dock inventory at their current location
  - Vessels can unload at any dock.
  - **All world state responses should now also report dock cargo inventories and vessel cargo inventories.**
  - Cargo enters via a new cargo_manifest field on init(); and on tick();, adding units to dock inventory.
- **Five new commands for dock operations:**
  - dock(); — Parks the vessel at its current location and sets planned state to DOCKED.
    - Valid if the committed state is IDLE.
    - Overwritable by set_destination(); before the next tick.
    - A pending set_destination(); can be overwritten by dock(); before the next tick.

    TBResolved:
    - What happens if a user commands dock after dock?
    - What happens if a user commands undock after dock?
  - undock(); — Returns vessel to IDLE at current location.
    - Valid if the committed state is DOCKED.

    TBResolved:
    - What happens if a user commands undock after undock?
    - What happens if a user commands dock after undock?
  - load(); — Moves one cargo from dock to vessel and updates planned state inventory
    - Valid when the committed state is DOCKED **and** inventory exists.
  - unload(); — Moves one cargo from vessel to dock and updates planned state inventory.
    - Valid when the committed state is DOCKED **and** inventory exists.
  - swap(); — Simultaneous load(); + unload();
    - Valid when the committed state is DOCKED **and** inventory exists.

TBResolved:

- What happens if a user commands undock after a load/unload/swap command?
- What happens if a user commands load/unload/swap after an undock command?

Dock Operation Rules

Dock Throughput

- dock_throughput[location] limits the total number of cargo operations that can be run across **all** vessels at that dock between ticks.
    - load(); counts as one operation
    - unload(); counts as one operation
    - swap(); counts as one operation
    - **Implications:** each dock's throughput is a budget across all docked vessel instances
    - Each load/unload/swap command is checked against this limit;
        - the command is rejected if the new pending state cannot be achieved within the dock_throughput limit.

    TBResolved:

        + What happens if a user calls load and unload on the same cargo?
            + How many operations does this count as?

Commands section

- init(); — Now requires dock_throughput. cargo_manifest is optional and initializes docks with cargo inventory.

```
{"command": "init", "parameters": {"travel_times": [[0,4,7],[4,0,3],[7,3,0]],
"dock_throughput": [2,4,1], "vessels": [{"id": "v1", "location": 0}, {"id": "v2",
"location": 2}], "cargo_manifest": [{"location": 0, "destination": 2, "quantity": 1},
{"location": 0, "destination": 1, "quantity": 1}, {"location": 2, "destination": 0,
"quantity": 1}]}}
{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "idle",
"location": 0, "cargo": []}, "v2": {"state": "idle", "location": 2, "cargo": []}},
"cargo": [{"location": 0, "destination": 2, "quantity": 1}, {"location": 0,
"destination": 1, "quantity": 1}, {"location": 2, "destination": 0, "quantity": 1}]}}
```

    - dock_throughput: N-length integer array. dock_throughput[i] is the max operations across all vessels at dock i per tick. All values positive.
    - cargo_manifest: optional.
        - Each entry is {location, destination, quantity}
            - location and destination both must be valid location indices.
            - quantity must be a positive integer.
            - If any entry is out of range, the command is rejected.

- tick(); — cargo_manifest is optional and adds cargo inventory to the newly committed state.


```
{"command": "tick", "parameters": {"cargo_manifest": [{"location": 0, "destination": 2, "quantity": 1}, {"location": 0, "destination": 1, "quantity": 1}, {"location": 2, "destination": 0, "quantity": 1}]}}
{"status": "ok", "payload": {"tick": 1, "vessels": {"v1": {"state": "idle", "location": 0, "cargo": []}, "v2": {"state": "idle", "location": 2, "cargo": []}}, "cargo": [{"location": 0, "destination": 2, "quantity": 1}, {"location": 0, "destination": 1, "quantity": 1}, {"location": 2, "destination": 0, "quantity": 1}]}}
```


    - cargo_manifest: optional.
    - Each entry is {location, destination, quantity}
        - location and destination both must be valid location indices.
        - quantity must be a positive integer.
        - If any entry is out of range, the command is rejected.

- dock(); — Parks the vessel at its current location and sets the planned state to docked.
    - Valid if the committed state is idle.
    - Overwritable by set_destination before the next tick.

e.g. — tick = 1, v1 committed state is idle at location 0:


```
{"command": "dock", "parameters": {"vessel_id": "v1"}}
{"status": "ok", "payload": {"tick": 1, "vessels": {"v1": {"state": "docked",
"location": 0, "cargo": []}, "v2": {"state": "idle", "location": 2, "cargo": []}},
"cargo": [{"location": 0, "destination": 2, "quantity": 1}, {"location": 0,
"destination": 1, "quantity": 1}]}}
```


On failure:


```
{"status": "error", "payload": {"message": "vessel v1 is not idle"}}
```

- undock(); — Returns vessel to idle at current location.
    - Valid if the committed state is docked.

e.g. — tick = 3, v1 committed state is docked at location 0:

```
{"command": "undock", "parameters": {"vessel_id": "v1"}}
{"status": "ok", "payload": {"tick": 3, "vessels": {"v1": {"state": "idle",
"location": 0, "cargo": [{"destination": 2, "quantity": 1}, {"destination": 1,
"quantity": 1}]}, "v2": {"state": "idle", "location": 2, "cargo": []}}, "cargo": []}}
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not docked"}}
```

- load(); — Moves one cargo from dock to vessel and updates planned state inventory.
    - Valid when the committed state is docked and inventory exists.

Example:

```
{"command": "load", "parameters": {"vessel_id": "v1", "destination": 2}}
{"status": "ok", "payload": {"tick": 2, "vessels": {"v1": {"state": "docked",
"location": 0, "cargo": [{"destination": 2, "quantity": 1}]}, "v2": {"state": "idle",
"location": 2, "cargo": []}}, "cargo": [{"location": 0, "destination": 1, "quantity":
1}]}}
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not docked"}}
{"status": "error", "payload": {"message": "transfer of 3 operations exceeds dock
throughput of 2 at location 0"}}
```

- destination: destination of the cargo unit to load from dock inventory

- unload(); — Moves one cargo from vessel to dock and updates the planned state inventory.
    - Valid when the committed state is docked and inventory exists.
    - If the cargo's destination matches the vessel's current location:
        - the cargo is delivered
        - the cargo is removed from the simulation rather than added to the dock inventory

```
{"command": "unload", "parameters": {"vessel_id": "v1", "destination": 2}}
{"status": "ok", "payload": {"tick": 12, "vessels": {"v1": {"state": "docked",
"location": 2, "cargo": [{"destination": 1, "quantity": 1}]}, "v2": {"state": "idle",
"location": 2, "cargo": []}}, "cargo": []}}
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not docked"}}
```

- destination: destination of the cargo unit to remove from the vessel

- swap(); — Simultaneous load + unload.
    - Valid when the committed state is docked and inventory exists.

e.g. — tick = 2, v1, committed state is docked at location 0 with cargo `[{"destination": 2, "quantity": 1}]`; and dock 0 has `[{"destination": 1, "quantity": 1}]`:

```
{"command": "swap", "parameters": {"vessel_id": "v1", "load_destination": 1, "unload_destination": 2}}
{"status": "ok", "payload": {"tick": 2, "vessels": {"v1": {"state": "docked", "location": 0, "cargo": [{"destination": 1, "quantity": 1}]}, "v2": {"state": "idle", "location": 2, "cargo": []}}, "cargo": [{"location": 0, "destination": 2, "quantity": 1}]}}
```

- load_destination: destination of the cargo unit to load from dock inventory
- unload_destination: destination of the cargo unit to remove from the vessel

- get_state(); — Response now includes dock cargo in world state and each vessel includes cargo.


```
{"command": "get_state", "parameters": {}}
{"status": "ok", "payload": {"tick": 3, "vessels": {"v1": {"state": "idle",
"location": 0, "cargo": [{"destination": 2, "quantity": 1}, {"destination": 1,
"quantity": 1}]}, "v2": {"state": "idle", "location": 2, "cargo": []}}, "cargo": []}}
```

Follow-on 2: Historical State Queries

Change from Base:

+ **All existing behavior is unchanged.**
+ Adds querying of past tick committed states.
+ init(); records committed state at tick 0;
+ each tick(); records **after** all transitions.
+ get_state_at(T) returns the committed state at T.
+ Intra-tick states are **not** queryable.
  TBResolved: What defines an intra-tick state?

Commands

get_state_at — Returns the committed state at a past tick. Errors if tick > current tick or tick < 0.

```
{"command": "get_state_at", "parameters": {"tick": 5}}
{"status": "ok", "payload": {"tick": 5, "vessels": {"v1": {"state": "transit", "from":
0, "to": 2, "arrives_at_tick": 7, "cargo": [{"destination": 2, "quantity": 1}]}, "v2":
{"state": "idle", "location": 2, "cargo": []}}, "cargo": []}}
```

On failure:

```
{"status": "error", "payload": {"message": "tick 99 is in the future (current tick:
36)"}}
```

- Tick must be:
  - <= current tick
  - >= 0

1.2) Requirements Decomposition

My decomposition uses INCOSE Systems Engineering Handbook and University of Michigan AEROSP388/488 Model-Based Systems Engineering (MBSE) practices, including tailoring to best align requirements definitions for the project scope. I am also referencing the MBSE "RFLP" framework. The requirements ID format for this project is as follows:

`[GROUP-LEVEL-ID]: Description`

Where **GROUP** must be one of:

- BASE → satisfies base engine needs
- FLW1 → satisfies follow-on 1 needs (dependency on BASE implemented)
- FLW2 → satisfies follow-on 2 needs (dependency on BASE and FLW1 implemented)

Where **LEVEL** must be one of:

- 0 → Derived from project document
- 1 → Derived from Level 0 or project document
    - Consist of black-box models or functional/behavioral standards, i.e. "System-level" requirements
- 2 → Derived from Level 1 or project document
    - Describe how subsystems (e.g. tick service, cargo service, vessel instruction handler) satisfy system needs
    - e.g. technical interfaces, budgets, technical architecture decisions
- 3 → Derived from Level 2 or project document
    - Describe specific components (e.g. datatypes, classes, algorithms) that ultimately sum to the capabilities required of their parent level 2 requirement

- Additional levels have been deemed unnecessary for this project's scope.

Where **ID** must be:

- An integer >= 0
- Unique to its group
- Unique to its level
- e.g. BASE-1-01 and FLW1-1-01 may coexist
- e.g. BASE-1-01 and BASE-2-01 may coexist
- e.g. BASE-1-01 and BASE-1-01 CANNOT coexist

- For traceability, requirements may also contain contextual information such as "derived-from" relationships, rationale, or assumptions.

Please see the next page for my requirements decomposition.

Level 0 Requirements

[BASE-0-00]: The fleet simulation engine shall utilize a discrete-time simulation.
- Derived from document

[BASE-0-01]: The fleet simulation engine shall receive all commands externally.
- Derived from document

[BASE-0-02]: The fleet simulation engine shall track vessel states at each point in time.
- Derived from document

[BASE-0-03]: The fleet simulation engine shall report vessel states at each point in time.
- Derived from document

[BASE-0-04]: The fleet simulation engine shall utilize JSON for the input-output interface.
- Derived from document

[BASE-0-05]: The fleet simulation engine shall produce one line of output per command.
- Derived from document

[BASE-0-06]: The fleet simulation engine shall process commands in their given order.
- Derived from document

[BASE-0-07]: The fleet simulation engine shall include separate planned and committed states for each vessel.
- Derived from document

[BASE-0-08]: The fleet simulation engine shall be consistently executable across different systems.
- Derived from document, specifically from “A way to run and test [the project].”

[FLW1-0-00]: The fleet simulation engine shall support cargo and dock operations.
- Derived from document

[FLW2-0-00]: The fleet simulation engine shall support querying of past tick committed states.
- Derived from document

[FLW2-0-01]: The fleet simulation engine shall preserve base and follow-on 1 behaviors.
- Derived from document

Level 1 Requirements

[BASE-1-00]: The fleet simulation engine shall report successes
- Derived from document

[BASE-1-01]: The fleet simulation engine shall report errors
- Derived from document

[BASE-1-02]: Commands shall take effect in the order received.
- Derived from document

[BASE-1-03]: The Planned State shall be updated before the response is written.
- Derived from document

[BASE-1-04]: The fleet simulation engine shall write outputs to stdout.
- Derived from document

[BASE-1-05]: The fleet simulation engine shall read newline-delimited JSON inputs from stdin.
- Derived from document

[BASE-1-06]: Committed States shall describe (“snapshot”) their respective Planned States at a tick.
- Derived from document

[BASE-1-07]: Planned States shall describe the expected state of the world since the last tick applied to the Committed State.
- Derived from document

[BASE-1-08]: Planned States shall update immediately with each command.
- Derived from document

[BASE-1-09]: Committed States shall become their respective Planned State when tick is called.
- Derived from document

[BASE-1-10]: Each tick command shall advance the simulation by one time step.
- Derived from document

[BASE-1-11]: The init command shall be a one-time, first command which returns the world state at tick 0.
- Derived from document

[BASE-1-12]: The set_destination command shall command an idle vessel to travel to a **new** location.
- Derived from document

- Keyword “new” introduced to restrict same-location traveling
    - Decided because set_destination implies travel.

[BASE-1-13]: The get_state command shall return the planned state.

- Derived from document

[BASE-1-14]: The fleet simulation engine shall be dockerized for portable, consistent execution.

- Derived from **BASE-0-08**

[BASE-1-15]: The fleet simulation engine shall utilize a language which supports portable, consistent execution.

- Derived from **BASE-0-08**

[FLW1-1-00]: Each location in the fleet simulation engine shall have a dock with cargo inventory.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-01]: Each location in the fleet simulation engine shall limit total cargo operations per tick through a throughput budget.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-02]: Vessels with inventory shall load from dock inventory at their current location.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-03]: Vessels with inventory shall be able to unload at any dock.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-04]: The fleet simulation engine world state responses shall report dock cargo inventories and vessel cargo inventories.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-05]: Cargo units shall be enterable into dock inventories.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-06]: An idle-state vessel shall be dockable.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-07]: Planned docking shall be overwritable by set_destination before the next tick.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-08]: Planned set_destination shall be overwritable by dock before the next tick.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-09]: A docked vessel with inventory shall be loadable.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-10]: A docked vessel with inventory shall be unloadable.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-11]: A docked vessel shall be undockable.

- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-12]: A docked vessel shall report an error if commanded to dock.

- Derived from document as a decision left to the implementer.
- Rationale: Dock after dock is redundant/idempotent regardless of planned or committed state... the second dock is effectively “do nothing”. An additional “do nothing” command consumes resources that we could invest elsewhere, i.e. we achieve the same functionality through not commanding at all. We should therefore inform the user that we’re already planning to dock through an error.
- Please see **FLW1-3-14** for planning undock and then commanding dock.
- Satisfies **FLW1-0-00**

[FLW1-1-13]: An undocked vessel shall report an error if commanded to undock (i.e. planned state $\neq$ docked).

- Derived from document as a decision left to the implementer.
- Rationale: Identical to **FLW1-1-12** rationale where we want to avoid redundant operations. An undocked vessel has nothing to undock from.
- Please see **FLW1-3-14** for planning undock and then commanding dock.
- Satisfies **FLW1-0-00**

[FLW1-1-14]: A swap command shall perform a simultaneous load and unload for a docked vessel with inventory.
- Derived from document
- Satisfies **FLW1-0-00**

[FLW1-1-15]: Calling undock on a vessel after load/swap/unload commands shall override the planned state with undock.
- Derived from document as a decision left to the implementer.
- Rationale:
    - Keeping load/swap/unload all in the same requirement because they are all the same operation in this context.
    - An assumption is made here: tick is not yet called. Because tick is not called, the vessel only plans to load/swap/unload, but undocking is also a valid command for a docked vessel. Therefore, we overwrite the planned cargo operation with undocking.
- Satisfies **FLW1-0-00**

[FLW1-1-16]: Calling load/swap/unload commands after undock shall override the planned state with load/swap/unload.
- Derived from document as a decision left to the implementer.
- Rationale:
    - Keeping load/swap/unload all in the same requirement because they are all the same operation in this context.
    - An assumption is made here: tick is not yet called. Because tick is not called, the vessel plans to undock but is currently docked nonetheless. Because the vessel is docked, cargo operations are valid commands.
- Satisfies **FLW1-0-00**

[FLW1-1-17]: The get_state command shall include relevant dock and cargo operations information in the returned planned state.
- Derived from document

[FLW2-1-00]: init shall record the committed state at tick 0.
- Derived from document
- Satisfies **FLW2-0-00**

[FLW2-1-01]: each tick shall record world state after all transitions.
- Derived from document
- Satisfies **FLW2-0-00**

[FLW2-1-02]: the command get_state_at(T) shall return the committed state at a valid T.

- Derived from document
- Satisfies **FLW2-0-00**

[FLW2-1-03]: Intra-tick states shall be non-queryable.

- Derived from document
- Satisfies **FLW2-0-00**

Level 2 Requirements

[BASE-2-00]: The engine shall report an error if any command is executed before init.

- Derived from document
- Satisfies **BASE-1-11**

[BASE-2-01]: The engine shall report an error if init is not called exactly once.

- Derived from document
- Satisfies **BASE-1-11**

[BASE-2-02]: The engine shall report an error if set_destination is called for a vessel whose committed state is not idle.

- Derived from document
- Satisfies **BASE-1-12**

[BASE-2-03]: The engine shall report an error if set_destination is called with the vessel's current location as a destination.

- Derived from **BASE-1-12**
- Satisfies **BASE-1-12**

[BASE-2-04]: The engine shall report an error if set_destination is called for a vessel in transit.

- Derived from document
- Satisfies **BASE-1-12**

[BASE-2-05]: The engine shall report an error if set_destination is called for an unknown vessel_id.

- Derived from document
- Satisfies **BASE-1-12**

[BASE-2-06]: The engine shall report an error if set_destination is called for a destination out of range.

- Derived from document
- Satisfies **BASE-1-12**

[BASE-2-07]: The fleet simulation engine shall be dockerized using a multi-stage build.

- Derived from **BASE-1-14**

- This keeps our container image small and reduces attack surface by minimizing the overall amount of software packaged.

[BASE-2-08]: The fleet simulation engine shall utilize Rust for implementation.
- Derived from **BASE-1-15**
- Choosing Rust over C++ for memory safety and reliability
    - NOTE: I have minimal technical experience in Rust. I am most experienced with C/C++. Therefore, I'd need to connect with engineers familiar with both languages for risk management / trade studies on this topic.

[BASE-2-09]: The set_destination command shall override previous set_destination commands.
- Derived from document
- Satisfies **BASE-1-12**

[BASE-2-10]: The set_destination command shall update the Planned State.
- Derived from document
- Satisfies **BASE-1-12**

[BASE-2-11]: A vessel that arrives at tick T shall report IDLE state at tick T.
- Derived from document
- Satisfies **BASE-1-12**

[BASE-2-12]: A vessel that arrives at tick T shall be able to be commanded in the same tick T.
- Derived from document
- Satisfies **BASE-1-12**

[FLW1-2-00]: Cargo additions to docks shall be interfaced through a "cargo_manifest" field on init and tick.
- Derived from document
- Satisfies **FLW1-1-05**

[FLW1-2-01]: Each location shall use the N-length integer array "dock_throughput" datatype to limit the total number of cargo operations per-tick (N defined as a positive integer).
- Derived from document
- Refines **FLW1-1-01**
- Satisfies **FLW1-1-01**

[FLW1-2-02]: cargo shall be represented only by its destination.
- Derived from document

- Refines **FLW1-1-00**
- Satisfies **FLW1-1-00**

[FLW1-2-03]: cargo_manifest shall be an optional argument for init which initializes a dock with cargo inventory.
- Derived from document
- Satisfies **FLW1-1-00**

[FLW1-2-04]: cargo_manifest shall be an optional argument for tick which adds cargo inventory to a dock.
- Derived from document
- Satisfies **FLW1-1-00**

[FLW1-2-05]: cargo_manifest entries shall take on the format “{location, destination, quantity}”
- Derived from document
- Satisfies **FLW1-1-00**

[FLW1-2-06]: Out-of-range cargo_manifest entries shall cause the parent command to be rejected with error reporting.
- Derived from document
- Satisfies **FLW1-1-00**

[FLW1-2-07]: Commanding dock on a non-idle vessel shall report an error.
- Derived from document
- Satisfies **FLW1-1-06**

[FLW1-2-08]: Calling load on an undocked vessel shall report an error.
- Derived from document
- Refines **FLW1-1-09**
- Satisfies **FLW1-1-09**

[FLW1-2-09]: Calling load on a vessel with no inventory shall report an error.
- Derived from document
- Refines **FLW1-1-09**
- Satisfies **FLW1-1-09**

[FLW1-2-10]: Calling unload on an undocked vessel shall report an error.
- Derived from document
- Refines **FLW1-1-10**
- Satisfies **FLW1-1-10**

[FLW1-2-11]: Calling unload on a vessel without inventory shall report an error.

- Derived from document
- Refines **FLW1-1-10**
- Satisfies **FLW1-1-10**

[FLW1-2-12]: Upon calling unload, if cargo destination matches vessel current location, the cargo shall be removed from the simulation (rather than added to dock inventory).

- Derived from document
- Refines **FLW1-1-10**
- Satisfies **FLW1-1-10**

[FLW1-2-13]: Upon calling unload, if cargo destination does not match vessel current location, the cargo shall be added to dock inventory.

- Derived from **FLW1-1-10**
    - Rationale: Inferred as the case where cargo mismatches dock location but is still unloaded.
- Refines **FLW1-1-10**
- Satisfies **FLW1-1-10**

[FLW1-2-14]: The unload command shall have a valid destination parameter.

- Derived from document.
- Refines **FLW1-1-10**
- Satisfies **FLW1-1-10**

[FLW1-2-15]: Calling swap on an undocked vessel shall return an error.

- Derived from document.
- Refines **FLW1-1-14**
- Satisfies **FLW1-1-14**

[FLW1-2-16]: Calling swap on a vessel without inventory shall return an error.

- Derived from document.
- Refines **FLW1-1-14**
- Satisfies **FLW1-1-14**

[FLW1-2-17]: swap parameter “load_destination” shall be a valid location.

- Derived from document.
- Refines **FLW1-1-14**
- Satisfies **FLW1-1-14**

[FLW1-2-18]: swap parameter “unload_destination” shall be a valid location.

- Derived from document.

- Refines **FLW1-1-14**
- Satisfies **FLW1-1-14**

[FLW2-2-00]: A valid T shall be defined as a tick greater than or equal to 0 and less than or equal to the current tick.
- Derived from document
- Refines **FLW2-1-02**
- Satisfies **FLW2-1-02**

[FLW2-2-01]: An invalid T shall result in an error being reported.
- Derived from document
- Refines **FLW2-1-02**
- Satisfies **FLW2-1-02**

Level 3 Requirements

[BASE-3-00]: The engine shall report “{"status": "ok", "payload": <WorldState>}” for successes
- Derived from document
- Satisfies **BASE-1-00**

[BASE-3-01]: The engine shall report “{"status": "error", "payload": {"message": "<reason>"}}” for errors
- Derived from document
- Satisfies **BASE-1-01**

[BASE-3-02]: set_destination’s arrives_at_tick shall be defined as “arrives_at_tick = T + travel_times[current_location][destination]”
- Derived from document
- Satisfies **BASE-1-12**

[BASE-3-03]: The “travel_times” datatype shall be defined as an N-by-N matrix where N is an integer.
- Derived from document
- Satisfies **BASE-1-12**

[BASE-3-04]: travel_times[i][j] shall return the ticks to travel from point i to point j.
- Derived from document
- Satisfies **BASE-1-12**

[BASE-3-05]: travel_times shall have a diagonal of 0.
- Derived from document
- Satisfies **BASE-1-12**

[BASE-3-06]: travel_times’ off-diagonal values shall be positive integers

- Derived from document
- Satisfies **BASE-1-12**

[BASE-3-07]: The “vessels” datatype shall be a list of {id, location}.

- Derived from document

[BASE-3-08]: The error payload shall include a message stating a redundant (destination = current location) destination.

- Derived from **BASE-2-03**
- Satisfies **BASE-2-03**

[FLW1-3-00]: The load command shall count as one operation.

- Derived from document
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-01]: The unload command shall count as one operation.

- Derived from document
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-02]: The swap command shall count as one operation.

- Derived from document
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-03]: For each location, an integer-valued “planned cargo operation” cost shall be updated per-cargo operation (i.e. updating accordingly for new cargo commands AND for overwrites).

- Derived from **FLW1-2-01**
    - Rationale: A “net-cost” value per-location facilitates checking by letting us add the latest cargo op’s cost to it and then comparing with throughput. For example:
        - load → add 1 to cost → cost **exceeds** throughput → reject command / report error for specific vessel.
        - swap → add 1 to cost → cost **does not exceed** throughput → accept vessel state update.
        - load → add 1 to cost → change to undock → sub 1 from cost → check is unnecessary, vessel state always updates to undock as per undock rule.
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-04]: For a vessel with inventory, a cargo operation command which causes its docked location's "planned cargo operation" cost to exceed throughput shall result in the command being rejected.
- Derived from document
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-05]: For a vessel with inventory, a cargo operation command which causes its docked location's "planned cargo operation" cost to exceed throughput shall result in an error being reported.
- Derived from document json example
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-06]: Calling load and unload on the same cargo in either order shall overwrite the previous planned command with the new command.
- Derived from document as a decision left to the implementer.
    - Rationale: Although a simultaneous load/unload is identical to swap, the semantics of calling load and unload prior to tick imply the overwriting of the previous instruction.
- To clarify, performing a new cargo command will also re-run checks for operation cost, by **FLW1-3-04**. This is a byproduct of architecture and justified for flexibility: we may want to change operation cost for load, unload, and/or swap later.
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-07]: dock_throughput[i] shall return the max operations ("throughput") across all vessels at dock i per tick.
- Derived from document
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-08]: dock_throughput[i] shall take on exclusively positive values.
- Derived from document
- Refines **FLW1-2-01**
- Satisfies **FLW1-2-01**

[FLW1-3-09]: The cargo_manifest "location" parameter shall be a valid location index.
- Derived from document
- Satisfies **FLW1-2-05**

[FLW1-3-10]: The cargo_manifest “destination” parameter shall be a valid location index.

- Derived from document
- Satisfies **FLW1-2-05**

[FLW1-3-11]: The cargo_manifest “quantity” parameter shall be a positive integer.

- Derived from document
- Satisfies **FLW1-2-05**

[FLW1-3-12]: swap parameter “load_destination” shall return an error if the argument is not a valid destination index.

- Derived from **FLW1-2-17.**
- Refines **FLW1-2-17**
- Satisfies **FLW1-2-17**

[FLW1-3-13]: swap parameter “unload_destination” shall return an error if the argument is not a valid destination index.

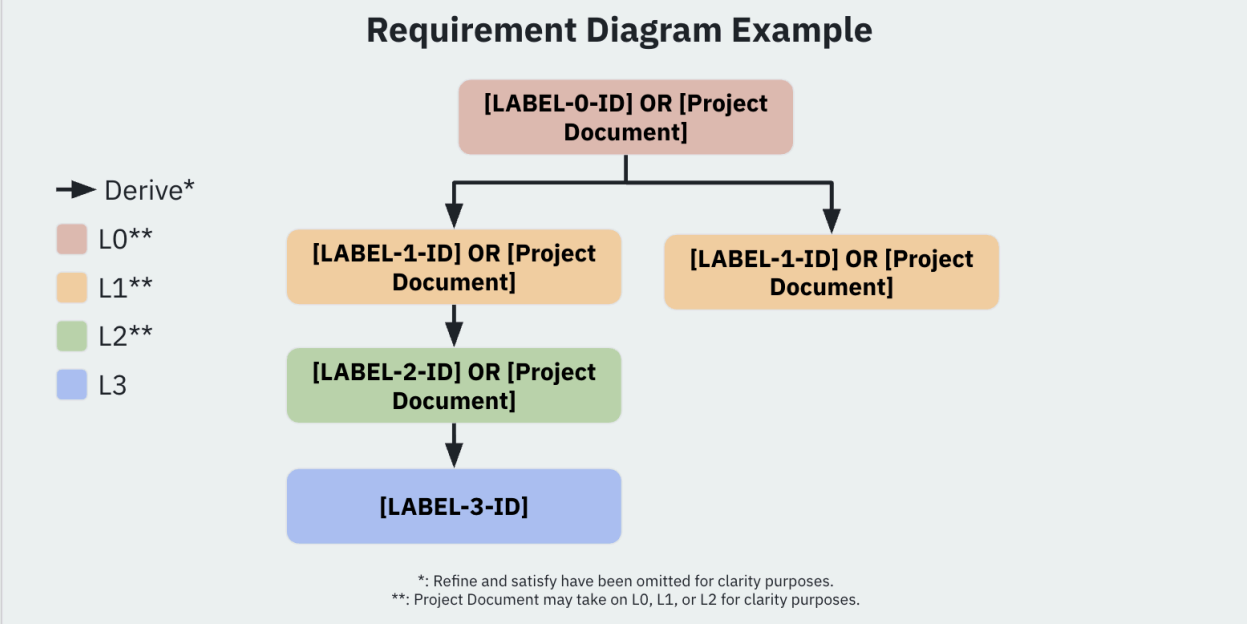
- Derived from **FLW1-2-18.**
- Refines **FLW1-2-18**
- Satisfies **FLW1-2-18**

[FLW1-3-14]: Commanding dock after commanding undock for a docked vessel shall return the planned state to “docked”.

- Derived from **FLW1-2-07.**
- Refines **FLW1-2-07**
- Satisfies **FLW1-2-07**

1.2.1) Google Slides Supplemental Illustration

Attached in my email response is a supplemental slide deck for this design document. The purpose of this deck is to visually arrange requirements to enhance readability. Below is an example of how the deck organizes requirements:



Implementation Plan

2.1) Trade Study

One trade study has been performed using the Pugh Method. The Pugh Method was chosen for its convenience and for the scope of this project. Only one trade study was performed because the project document is already technically specific.

Study 1: Software Languages

	C	**C++**	**Rust**	**Python**
Familiarity	S	S	-	-
Efficiency	S	S	S	-
Memory Safety	-	-	+	+
Ecosystem	-	S	+	+
Sum of Positives	0	0	2	2
Sum of Sames	2	3	1	0
Sum of Negatives	2	1	1	2
Chosen?	No	No	**Yes**	No

2.2) Workflow Overview

The sequence for implementation is as follows:

1. Satisfy **BASE** requirements
2. Satisfy **FLW1** requirements
3. Satisfy **FLW2** requirements

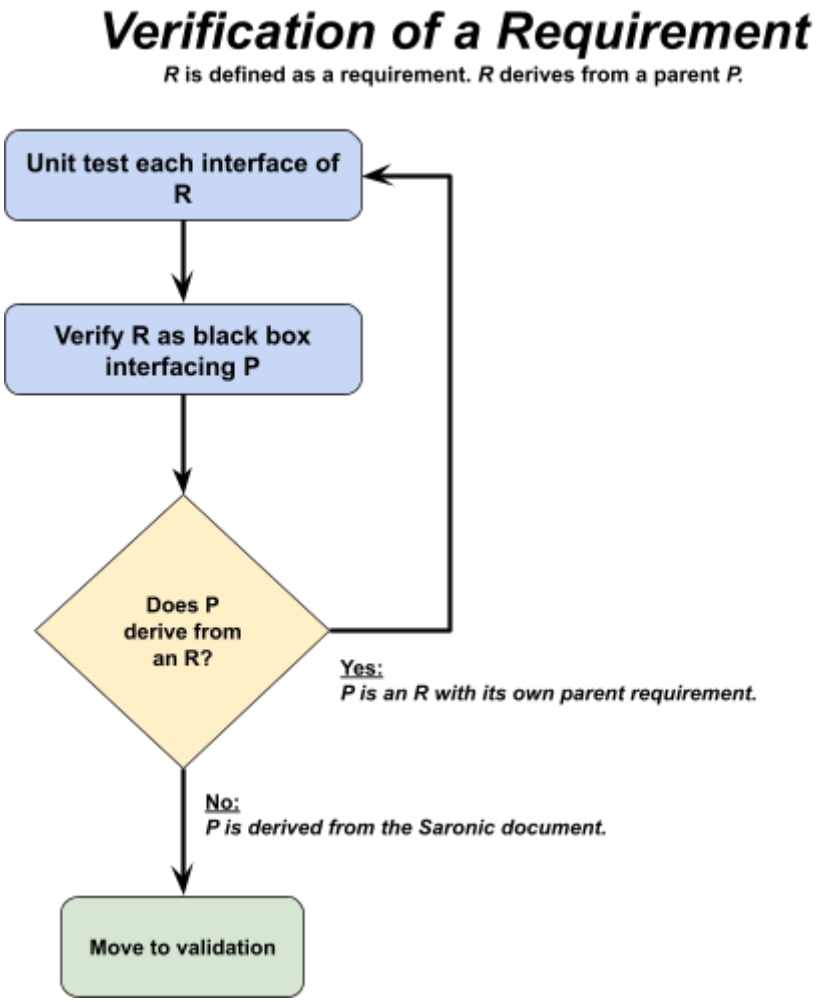
Where requirements are satisfied in order:

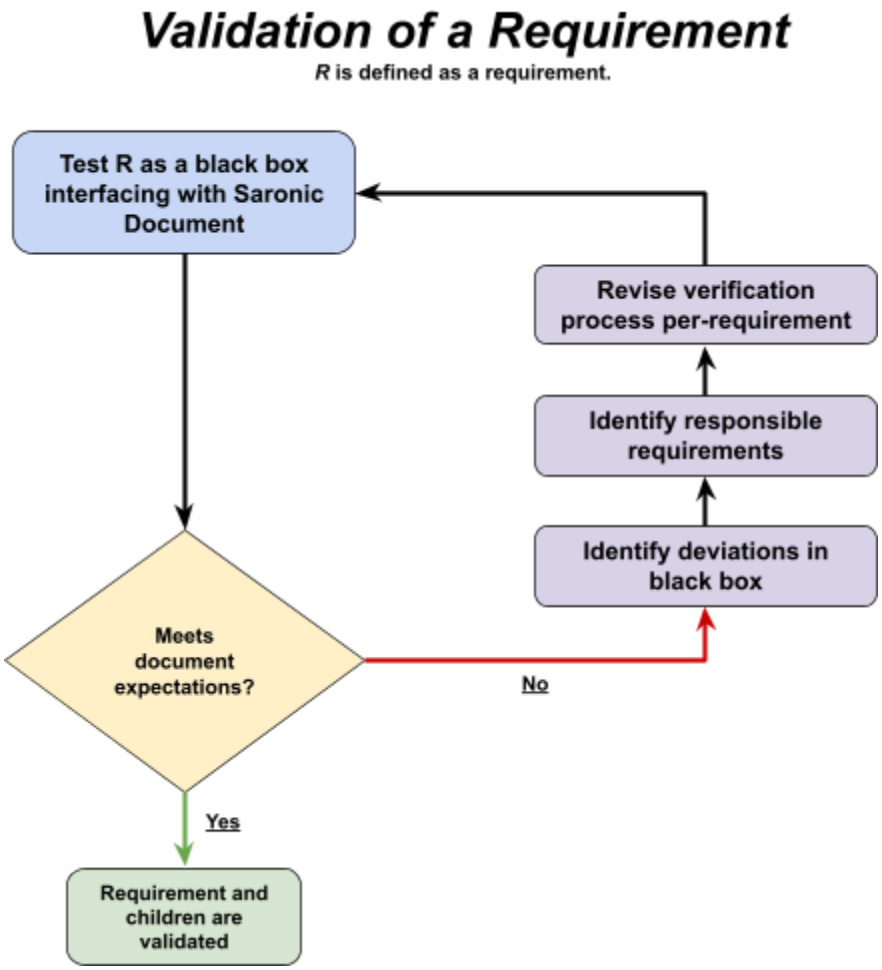
L3 → L2 → L1 → L0

As per the INCOSE V-Model framework. *The V-Model was chosen over the Spiral model, Arch model, etc. due to personal familiarity.*

2.3) Verification and Validation

To clarify, Verification and Validation ask separate questions. Verification asks, “Did we build the thing right?” while Validation asks, “Did we build the right thing?”. Therefore, the two testing processes are different in what they observe. Below are two diagrams that highlight the two processes:




Validation of a Requirement
R is defined as a requirement.
Test R as a black box interfacing with Saronic Document
Revise verification process per-requirement
Identify responsible requirements
Identify deviations in black box
Meets document expectations?
No
Yes
Requirement and children are validated

Post-Implementation Analysis

This section records my analysis of the project outcome as of October 19, 2026. This includes scalability implications, unresolved items that I need help with to confidently address, and lessons learned for future iterations.

3.1) Scalability

As the validation process concluded, Claude reported the following performance metric:

For *10* vessels*, 9* docks*: 1.3 KB / tick*. Therefore, *272 MB* over *200,000* ticks.

Then, as per my request, Claude stress-tested an arbitrary *1000* vessels and *50* docks (100x the original vessel quantity, ~5.6x the original dock quantity). The following worst case scenario was observed where each vessel held cargo for every dock:

For *1000* vessels, *50* docks: *~0.94 MiB* per tick

And the following was observed where each vessel held no cargo:

For *1000* vessels, *50* docks: *~0.16 MiB* per tick

Claude noted that commands take approximately *6* ms at this size. Claude also observed that each response is, at maximum, *1.77 MB* of JSON.

During a cargo operations performance test, Claude found that *50,000* loads took approximately *5* minutes and produced an output estimated at *50 GB*. Therefore, scalability has been registered as an unresolved item.

3.2) Unresolved Items

1. Responsible Engineer is Unfamiliar with Rust
    a. I have little experience in Rust, so I am unable to verify the implementation with the scrutiny that I could in C++. However, the security and ecosystem benefits that Rust offers here mitigate an otherwise massive attack surface (Memory Safety Vulnerabilities). Furthermore, my solo review of a C++ implementation would take a long time and certainly ensure less than Rust's default safety features.
    b. **Therefore**, if this engine is expanded upon, I would seek aid from an engineer(s) with a strong Rust and C++ background. Their review would contribute to both project health and my own engineering judgement in the form of lessons learned and action items (e.g. Rust exercises and qualification metrics to gauge my own fluency).
2. Scalability
    a. As per 3.1, scalability is an issue. Notably, output size is more taxing than memory usage.
    b. **Therefore,** if this engine is expanded upon, I would perform research on scaling and optimization before preparing a new engineering design document to address scalability challenges. Additionally, I would reach out to engineers familiar with software infrastructure / database infrastructure for expertise, lessons learned, and action items.
3. Risk Mitigation Process
    a. Due to the scope of this project, I did not produce any formal risk mitigation plans, nor did I produce an FMEA table for analysis.
    b. **Therefore,** if this engine is expanded upon, I would want to immediately reach out to the project's/department's respective risk members for clarification on how risk mitigation shall be pursued moving forward.
4. Limited System Model
    a. This project was solely designed in this engineering design document and does not have a formal SysML / SysMLv2 model available for digital integration with other Saronic services.
    b. **Therefore,** if this engine is expanded upon, I would search for Saronic standards for information exchange and digital integration of solutions. To clarify why this is an unresolved item: my background is in Dassault Cameo / INCOSE / University of Michigan systems engineering practices. From prior engineering experience, isolated EDDs quickly become inefficient when managing solutions at scale.
5. Specification needs clarification at some points
    a. In "manual.md" (the project's manual on running and testing), there is a section labeled "Choices where the spec is silent". This lists several areas where specification was limited, so intent was inferred and then implemented.
    b. **Therefore,** if this engine is expanded upon, I would seek the producer of the specifications for clarification on the areas and approval/rejection of the inferred features.

3.3) Lessons Learned

The lessons learned section is limited as of October 19, 2026, due to not yet having stakeholder feedback. Therefore, entries here are all personal reflections.

1. Clear separation between assumptions and definitions is critical.
    a. Therefore, in my requirements decomposition, I have featured rationale in areas where a decision may be unclear.

2. High-quality communication is highly valuable for reviewers, including the same responsible engineer in the future. In other words, nobody wants to read a badly-formatted, jumbled, or vague engineering design document. If a qualified reader can't clearly reconstruct the thought process of the responsible engineer, then the document needs polishing and/or supplemental content.
    a. I have made my best attempt to achieve this within project scope. This is why I produced a supplemental slide deck for my requirements: visual relationships between requirements is vastly superior to a ~100-entry linear text list.

3. Garbage in, Garbage out, always!
    a. This lesson comes from beyond this project, but it's important to emphasize here because of the prevalence of AI. We wouldn't want to work with ill-defined, vague input as engineers, nor would we want to give it to others. Unless we want a garbage product. In the same way, giving an AI service poor context, no constraints, and a set of ill-defined outcomes will result in garbage.
    b. This lesson relates strongly to architecture. A poor architecture may be implemented flawlessly and what results is a… flawlessly-implemented poor architecture. Therefore, I invested most of my time in this project scrutinizing requirements and considering V&V methodology.